



BANGALORE INSTITUTE OF TECHNOLOGY

Autonomous Institute, Affiliated to VTU

Department of Information Science And Engineering

DBMS Mini Project Synopsis

On

“INVENTORY MANAGEMENT SYSTEM”

USN	NAME	BATCH	PHONE NO
1BI24IS131	Rohit Maiya M	C1	9900607239
1BI24IS137	Samarth Karnam	C1	9740662305
1BI24IS140	Sanskar Patil	C1	9743414950
1BI24IS142	Shailesh Nayak	C1	9108616035

Lab In-Charges:

Prof. Padmanabha J

Prof. Vidasree T K

Associate Professors

Dept. of ISE, BIT

2025-26

Remarks by the Faculty

Acceptance:

Modified:

Rejection:

Signature of the Lab In-Charges

TABLE OF CONTENTS

SL.NO	TITLE	PAGE NO.
1.	INTRODUCTION	3
2.	TECHNICAL STACK	4
3.	ER – DIAGRAM	5-7
4.	DATABASE SCHEMA	8-9
5.	SOFTWARE AND HARDWARE REQUIREMENTS	10
6.	APPLICATION MODULES	11
7	APPLICATIONS OF THE SYSTEM	12

Chapter 1

Introduction

1.1. Overview

The Inventory Management System (IMS) is a digital solution designed to track the flow of goods from procurement to final sale. Efficient inventory management is critical for business success, as it prevents "stock-outs" and minimizes the capital tied up in excess stock. This project utilizes a Pure SQL backend for high-integrity data management, ensuring that business logic is handled directly at the database level for maximum reliability

1.2. Problem Statement

The reliance on traditional manual tracking and basic spreadsheets poses significant risks to modern business operations. Manual entry is inherently prone to human error, leading to discrepancies between recorded and actual physical stock. Furthermore, the absence of real-time monitoring results in delayed reordering, causing "out-of-stock" scenarios that directly impact revenue. Without a centralized relational database, businesses struggle with poor visibility, making it difficult to analyze historical stock movements or evaluate supplier performance. These inefficiencies create a bottleneck in the supply chain, necessitating a robust, automated solution to maintain data integrity and operational continuity..

1.3. Objectives

The primary goals of this system are:

- Real-time Tracking: Maintaining accurate, live counts of all warehouse items..
- Automated Alerts: Using SQL logic to trigger notifications when stock hits a "Reorder Point."
- Supplier Management: Linking products to specific vendors for streamlined purchasing.
- Transaction Logging: Maintaining an audit trail of all inward and outward stock movements

1.4. Methodology

The project follows a streamlined client-server architecture. The backend is powered by a Pure SQL environment, which handles core business logic, data normalization, and relational integrity via triggers and constraints. The frontend is developed using Python to facilitate user interaction and data visualization, ensuring a seamless interface for managing stock levels and viewing system reports.

Chapter 2

Technical Stack

This section outlines the primary tools and technologies used to develop the Inventory Management System. The architecture is designed to prioritize data integrity on the backend while maintaining a flexible, scriptable interface on the frontend..

2.1. Backend:MySQL (Pure SQL)

The backend serves as the "brain" of the system. Unlike traditional applications that rely on middle-tier application servers for logic, this project leverages the power of Pure SQL within the MySQL environment.

- Role: Database Management & Business Logic.
- Key Features:
 - Data Integrity: Utilizes Primary and Foreign Keys to ensure relational consistency across products, suppliers, and transactions.
 - Stored Procedures & Triggers: Complex business rules, such as automatically updating stock levels after a sale or checking reorder thresholds, are handled directly within the database
 - Normalization: Data is organized into normalized tables to reduce redundancy and prevent data anomalies.
 - ACID Compliance: Ensures that all inventory transactions (inward/outward) are processed reliably

2.2. Frontend: Python

Python is employed as the frontend tool to create the user interface and facilitate communication with the MySQL backend. It provides a bridge between the user's actions and the database execution

.

- Key Features:
 - Database Connectivity: Uses specialized libraries (such as mysql-connector-python or PyMySQL) to execute SQL queries and fetch results.
 - Input Processing: Captures user data from forms (e.g., adding a new product or updating supplier info) and sends it to the MySQL server.
 - Data Presentation: Formats the raw data retrieved from SQL into a readable format for the end-user.

Chapter 3

ER Diagram

3.1. Description of Components

Entity 1: Supplier (Rectangle)

Attributes (Ovals):

- SupplierID (Underlined as it is the Primary Key)
- SupplierName
- Phone Numbers (Double Oval as it is a Multi-valued attribute)
- Email
- Address (Composite attribute that branches into three simple ovals: Street, City, and Zip)

Entity 2: Product (Rectangle)

Attributes (Ovals):

- ProductID (Underlined as it is the Primary Key)
- Name
- Category
- StockQuantity
- Price

Entity 3: Warehouse (Rectangle)

Attributes (Ovals):

- WarehouseID (Underlined as it is the Primary Key)
- Location
- Capacity
- ManagerName
- Available Space (Dashed Oval as it is a Derived attribute)

Entity 4: User (Rectangle)

Attributes (Ovals):

- UserID (Underlined as it is the Primary Key)
- FName
- LName
- Phone Numbers (Double Oval as it is a Multi-valued attribute)
- Address (Composite attribute that branches into three simple ovals: Street, City, and Zip)

Entity 5: Transaction (Rectangle)**Attributes (Ovals):**

- TransactionID (Underlined as it is the Primary Key)
- Type
- Quantity
- TransactionDate
- TotalAmount (Dashed Oval as it is a Derived attribute)

3.2 Description of Relationships**Relationship 1: Supplies (Diamond)**

- Connects Supplier and Product
- Cardinality: 1 : M
- One Supplier can supply many Products

Relationship 2: Stored In (Diamond)

- Connects Product and Warehouse
- Cardinality: M : 1
- Many Products can be stored in one Warehouse

Relationship 3: Performs (Diamond)

- Connects User and Transaction
- Cardinality: 1 : M
- One User can perform many Transactions

Relationship 4: Involved (Diamond)

- Connects Product and Transaction
- Cardinality: M : N
- Many Products can be involved in many Transactions

3.3. Visual Layout for Drawing

- Center/Top-Center: Place the Product rectangle
- Left Side: Place the Supplier rectangle, with a Diamond labeled "Supplies" connecting it to the Product entity.
- Top-Right Side: Place the Warehouse rectangle, with a Diamond labeled "Stored In" connecting it back to the Product entity.
- Bottom-Right Side: Place the Transaction rectangle. Connect it up to the Product entity with a Diamond labeled "Involved".
- Bottom-Left Side: Place the User rectangle, with a Diamond labeled "Performs" connecting it across to the Transaction entity.
- Attribute Placement: Branch the Ovals out from each respective Rectangle like spokes on a wheel, making sure the composite Address branches into its sub-parts, the PhoneNumbers oval is double-lined, and the AvailableSpace oval is dashed.

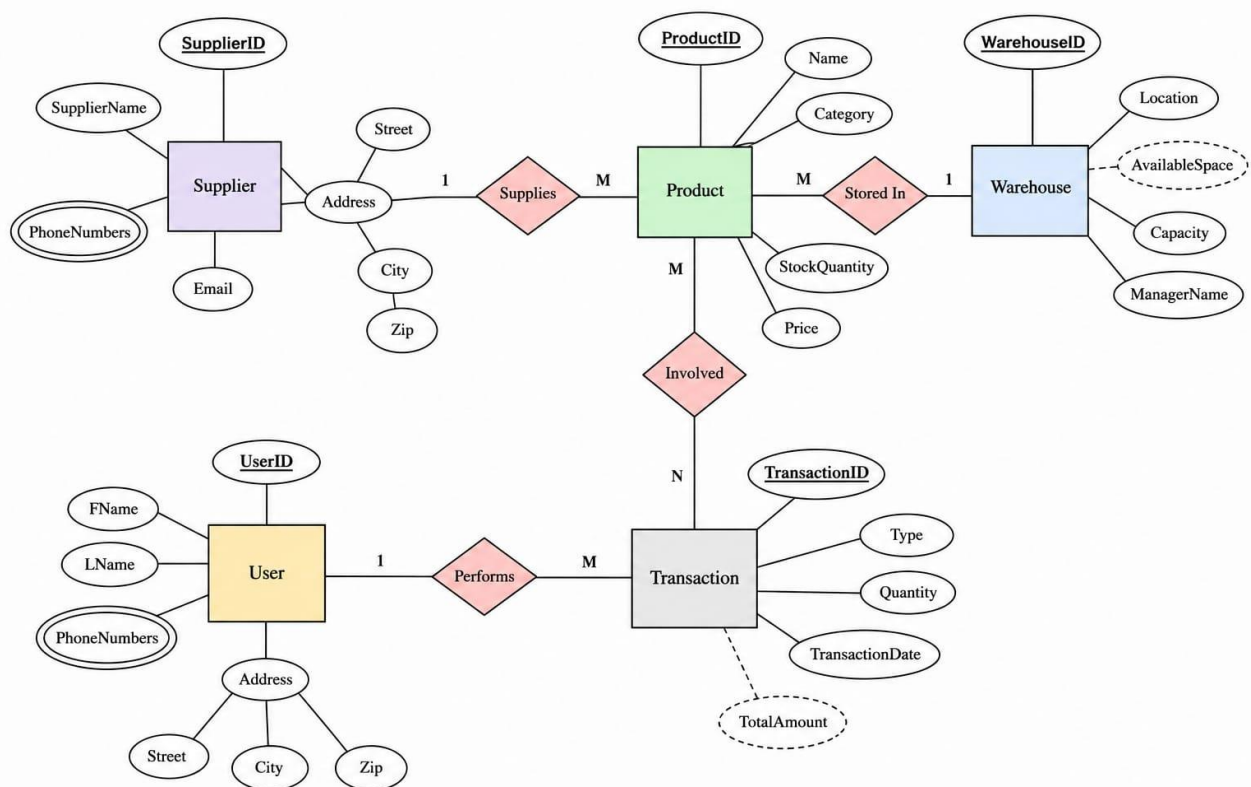


Fig 1.1 ER Diagram

Chapter 4

Database Schema

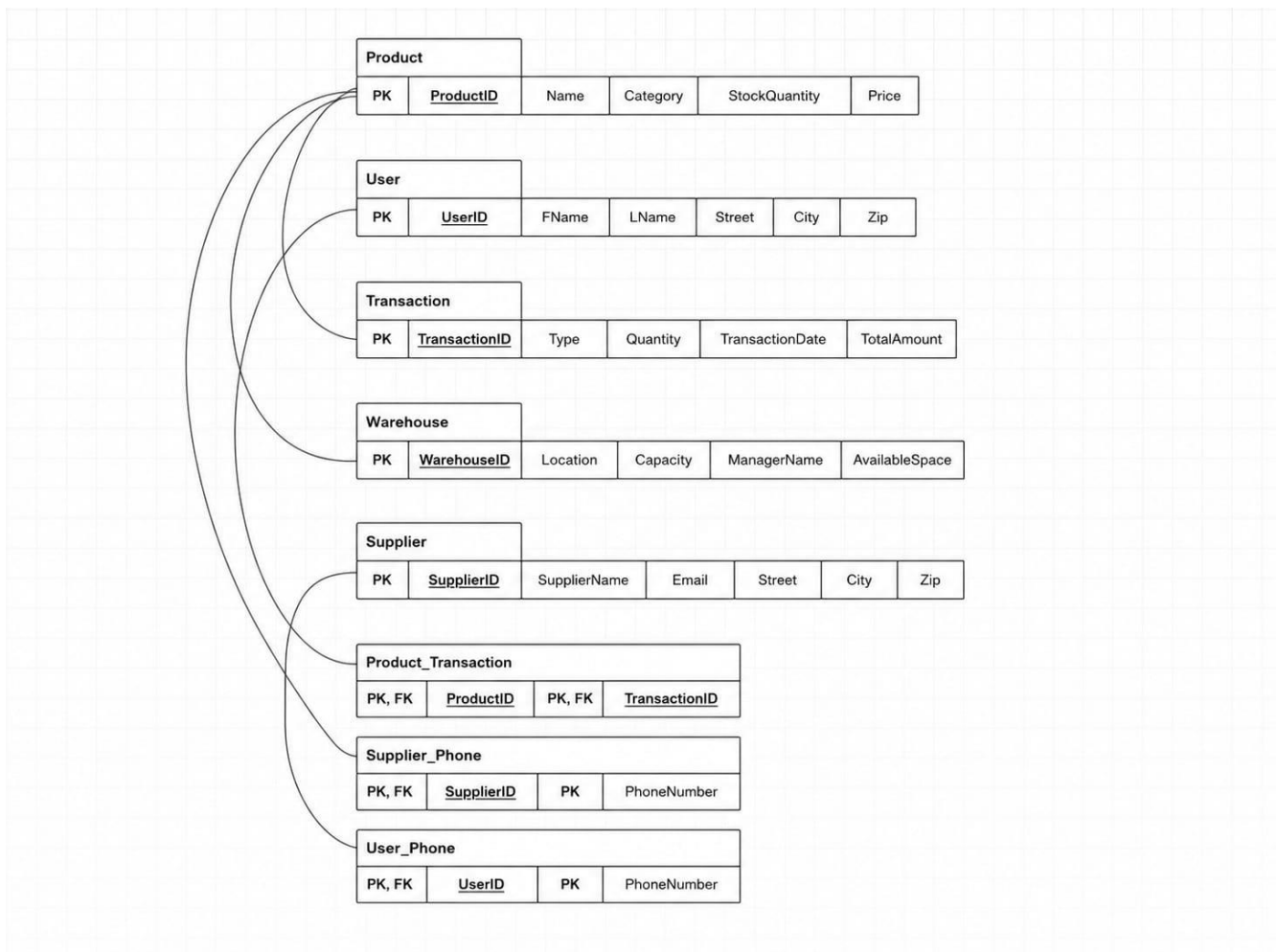
4.1. Architectural Overview

The backbone of this Inventory Management System is a Relational Database Management System (RDBMS). By utilizing Pure SQL, the system ensures data integrity through strict adherence to ACID properties (Atomicity, Consistency, Isolation, and Durability) using database-level constraints, triggers, and relational logic rather than relying on middleware. The schema is designed using Third Normal Form (3NF) normalization. This structure allows for complex queries, efficiently handles composite attributes by flattening them into specific columns, maps multi-valued attributes into normalized child tables, and dynamically calculates derived attributes (such as Available Space) using SQL views or calculated columns. This ensures that every piece of information is stored in a single, logical location, eliminating redundancy as the inventory grows.

4.2. Relational Model & Notation

The logical structure of the database is mapped from the conceptual ER diagram into the following relational tables. In this notation, Primary Keys (PK) are underlined, and Foreign Keys (FK) are italicized:

- The database manages products, users, suppliers, and warehouses.
- Suppliers store IDs, names, emails, and addresses.
- Products contain category, stock, and price details.
- Warehouses track location, capacity, and managers.
- Users store names and address information.
- Transactions record quantity, type, and amount.
- Supplier phone numbers are stored separately.
- User phone numbers use a separate table.
- Bridge tables handle many-to-many relationships.
- The schema follows 3NF for normalization

**Fig 1.2 IMS Schema Diagram**

Chapter 5

Software Requirements And Hardware Requirements

5.1. Hardware Requirements

To effectively develop and run the local database server, frontend interface, and development tools without performance bottlenecks, the following minimum hardware specifications are required:

- **Processor (CPU):** Intel Core i3 / AMD Ryzen 3 (or equivalent dual-core processor) minimum. An Intel Core i5 / AMD Ryzen 5 is recommended for smoother database query execution during testing.
- **Memory (RAM):** 4 GB minimum. 8 GB or higher is highly recommended to comfortably run the DBMS server, IDE, and web browser simultaneously.
- **Storage:** Minimum 5 GB of free disk space (Solid State Drive preferred) to accommodate the OS, IDE, MySQL Server, MySQL Workbench, and the project files.
- **Display:** 1366x768 screen resolution minimum for comfortably viewing the database schema diagrams and code side-by-side.

5.2. Software Requirements

The development environment is structured to ensure that all business logic is handled entirely within the relational database, using the frontend strictly for user interaction and data presentation.

- **Database Management System (DBMS):** MySQL (or PostgreSQL). This is the primary component where the project's core logic resides. All stock calculations, automatic reordering alerts, and data integrity constraints are managed through Pure SQL using Triggers, Stored Procedures, and Views.
- **Database Tooling:** MySQL Workbench. This tool is essential for designing the relational schema, managing the Entity-Relationship model, and writing/debugging the SQL scripts that drive the application.
- **Frontend Technologies:** HTML5, CSS3, and Vanilla JavaScript. These are used to create the user dashboard. JavaScript handles the presentation layer and the collection of user inputs, which are then passed directly to the SQL engine for processing.
- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code). Utilized for writing the frontend code and organizing the SQL script files.
- **Web Browser:** Google Chrome, Mozilla Firefox, or Microsoft Edge. Used to test the interface responsiveness and monitor the interaction between the browser and the SQL server.
- **Operating System:** Windows 10/11, macOS, or a standard Linux distribution (e.g., Ubuntu).

Chapter 6

Application Modules

6.1. Authentication & Security Module

The Authentication module serves as the gateway to the system, ensuring data security and accountability. Developed in Python, it provides a secure login interface where users must provide valid credentials to gain access. Behind the scenes, this module communicates with the MySQL user tables to verify identities. It also establishes the user session, ensuring that every subsequent action—such as adding stock or modifying a supplier—is tagged with a unique user ID for audit purposes.

6.2. Core Inventory Management Module

This is the primary operational hub of the application, designed to bridge the gap between the user and the database tables. Through Python-driven forms, users can register new products, categorize items, and set initial stock levels. The module includes a robust search and filter system that allows for the rapid retrieval of specific item data. When a user modifies a record, the frontend sends targeted SQL commands to the backend, where rela

6.3. Supplier & Procurement Module

To maintain a seamless supply chain, this module manages the directory of vendors and the logic of restocking. It provides a dedicated interface for viewing supplier contact details and linking specific products to their primary vendors. A key feature of this module is the "Reorder View," which fetches data from the SQL backend to highlight items that have fallen below their minimum thresholds. This allows the user to quickly identify which vendors need to be contacted for procurement without manually checking every stock level.

6.4. Transaction Tracking & Reporting Module

The Transaction module is responsible for the movement of goods, handling both "Inward" (procurement) and "Outward" (sales/dispatches) entries. The Python interface captures the quantity and timestamp of each movement and triggers the underlying SQL logic to update the master inventory table. Additionally, this module generates summary reports, translating raw rows of SQL transaction logs into readable tables. This provides a transparent audit trail, allowing managers to review historical movements and detect discrepancies in the warehouse flow.

Chapter 7

Applications of the System

The Inventory Management System (IMS) is designed with a versatile architecture that makes it adaptable across various industries. By leveraging a Pure SQL backend for data integrity and a Python frontend for accessibility, the system can be deployed in any environment where the precise tracking of goods and suppliers is essential.

7.1. Retail and General Merchandising

In the retail sector, the system is used to manage a diverse range of Stock Keeping Units (SKUs). It enables shop owners to track daily sales, monitor shelf levels in real-time, and ensure that popular items are never out of stock. The automated reorder alerts are particularly beneficial for retail, where high turnover rates require constant monitoring of inventory levels. Freelancer and Startup Co-Working Hubs

7.2. Healthcare and Pharmacy Management

The IMS is highly effective in pharmacies and clinics where the tracking of medicine batches and expiration dates is critical. The system ensures that high-value or life-saving drugs are consistently available by linking them to reliable suppliers. The high-integrity SQL backend ensures that sensitive stock data remains accurate and compliant with safety standards. Academic and Research Incubators

7.3. Hardware and Construction Warehousing

For industries dealing with heavy machinery, tools, and raw materials, the system provides a robust framework for managing bulk inventory. It helps warehouse managers track the movement of large quantities of items from the point of arrival to their dispatch for specific projects. The transaction logging feature creates a clear audit trail, reducing the risk of equipment loss or theft.

7.4. Small-Scale Manufacturing Units

Small manufacturing businesses can utilize the system to track raw materials and finished goods. By monitoring the consumption of materials during the production process, managers can predict when to order more supplies to avoid production halts. This ensures a lean manufacturing process where capital is not unnecessarily tied up in excess raw materials.